

## 모델연결하기

학습해 둔 모델(model.pkl)을 앱에 불러와,  
입력 → 예측 → 결과 시각화까지 완성합니다.

### Q1 큰 그림 — 학습은 따로, 예측은 앱에서

모델을 앱을 켤 때마다 새로 학습 하면 느리고 낭비예요. 그래서 미리 학습해 파일(model.pkl)로 저장해 두고, 앱은 그 파일을 불러와서 예측만 합니다.



학습·저장은 한 번, 앱은 불러와 예측만 반복.

"모델은 미리 구워서 파일로, 앱은 데워서 쏜다"

joblib이 모델을 파일로 굽고(dump), 다시 꺼내옵니다(load).

## Q2 모델 저장하고 불러오기 — joblib

joblib 은 학습한 모델을 파일로 저장(dump)하고 다시 불러오는(load) 도구입니다.

```
import joblib
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier().fit(X, y)  # 1) 학습
joblib.dump(model, "model.pkl")          # 2) 파일로 저장

# --- 나중에 앱에서 ---
model = joblib.load("model.pkl")          # 3) 파일에서 불러오기
model.predict(X_new)                      # 4) 예측
```

.pkl은 파이썬 객체를 통째로 담은 '피클' 파일이에요. 모델의 학습된 상태가 그대로 들어 있어, 불러오면 바로 예측할 수 있습니다.

## Q3 model.py — 로드와 예측을 함수로

실전에서는 모델 관련 코드를 `model.py` 로 따로 빼둡니다. 그러면 `app.py` 는 화면에만 집중할 수 있어요. (02\_MODEL\_ADD 구조)

```
# model.py
import joblib
import pandas as pd

def load_model(path="model.pkl"):
    return joblib.load(path)  # 모델 파일 불러오기

def predict(model, input_data):
    df = pd.DataFrame([input_data])  # 입력(dict) → 한 줄짜리 표
    prediction = model.predict(df)[0]  # 예측 클래스
    probability = model.predict_proba(df)[0]  # 각 클래스 확률
    return prediction, probability
```

왜 `pd.DataFrame([input_data])` 나면 — 모델은 학습할 때와 똑같은 표 모양의 입력을 원합니다. 사용자가 입력한 값들을 딕셔너리 → 한 줄짜리 표로 만들어 놓아야 `predict` 가 제대로 동작해요.

## Q4 모델은 한 번만 불러오기 — cache\_resource

02장에서 배웠듯 스트림릿은 버튼 한 번에 코드를 처음부터 다시 실행합니다(리런). 그때마다 모델 파일을 또 읽으면 느려요. @st.cache\_resource 를 붙이면 모델을 딱 한 번만 불러와 재사용 합니다.

```
import streamlit as st
from model import load_model, predict

@st.cache_resource          # 모델은 한 번만 로드(리런돼도 재사용)
def get_model():
    return load_model("model.pkl")

model = get_model()
```

**속도의 핵심입니다.** 모델 파일이 크면 로딩이 오래 걸리는데, cache\_resource 덕분에 첫 로드 이후엔 즉시 예측할 수 있어요.

## Q5 입력 → 예측 → 결과 시각화

이제 폼으로 입력을 받아 예측하고, **metric-progress**로 결과를 보기 좋게 보여줍니다. (02\_MODEL\_ADD의 소득 예측 앱)

```
with st.form("prediction_form"):
    age = st.number_input("나이", 17, 90, 38)
    hours = st.number_input("주당 근무시간", 1, 100, 40)
    submitted = st.form_submit_button("소득 예측하기")

if submitted:
    pred, proba = predict(model, {"age": age, "hours.per.week": hours})
    st.metric("예측 결과", ">$50K" if pred==1 else "<=$50K")
    st.progress(float(proba[1]), text=f"고소득 확률 {proba[1]:.1%}")
```



### 예측 결과

예측 소득 구간

>\$50K

고소득 확률

73%

>\$50K 확률



예측 결과를 metric 카드와 progress 바로 시각화.

## Q6 실전에서 꼭 지킬 것

| 주 의         | 왜                                                                              |
|-------------|--------------------------------------------------------------------------------|
| 버전 일치       | 모델(.pkl)을 만든 <b>scikit-learn-lightgbm</b> 버전과 앱의 버전이 같아야 함. 다르면 로드 시 경고/오류     |
| 입력 컬럼 이름·순서 | 학습할 때 쓴 <b>열 이름과 똑같이</b> 넣어야 함(예: <code>education.num</code> ). 하나라도 다르면 예측 실패 |
| 인코딩 값       | 글자 항목을 숫자로 학습했다면, 앱 입력도 <b>같은 숫자 규칙</b> 으로 넣어야 함                               |

가장 흔한 실수는 버전 불일치입니다. 01장의 `requirements.txt` 를 모델 학습 환경과 똑같이 맞추면 대부분 해결돼요.

## Q7 한 장 정리

| 단 계       | 코 드                                                 |
|-----------|-----------------------------------------------------|
| 학습 → 저장   | <code>joblib.dump(model, "model.pkl")</code>        |
| 불러오기(한 번) | <code>@st.cache_resource + joblib.load</code>       |
| 입력 → 표    | <code>pd.DataFrame([input_data])</code>             |
| 예측        | <code>model.predict · model.predict_proba</code>    |
| 결과 시각화    | <code>st.metric · st.progress · st.bar_chart</code> |

### 이것만 기억하면 됩니다

- ① 모델은 미리 학습해 `.pkl` 로 저장, 앱은 불러와 예측만
- ② `@st.cache_resource` 로 모델을 한 번만 로드
- ③ 입력은 학습 때와 같은 표 모양( `pd.DataFrame([...])` )으로
- ④ 버전·컬럼 이름을 학습 환경과 똑같이 맞춘다

## Q8 단계별로 구축하기

붓꽃(iris) 예측기를 **학습·저장** → **예측 앱** → **결과 시각화** 순서로 구축합니다. STEP 1은 주피터(또는 `train.py`)로 한 번, STEP 2·3은 `app.py`에 붙여 `docker compose up`으로 확인하세요.

**코드 해석** ▼을 누르면 한 줄씩 쉽게 풀어 줍니다.

### STEP 1 학습 → 저장 (train.py) — model.pkl 만들기

```
import joblib
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier

iris = load_iris()
model = RandomForestClassifier(random_state=0).fit(iris.data, iris.target)

# 모델 + 메타정보(특성명·품종명)를 함께 저장하면 앱에서 쓰기 편함
joblib.dump(
    {"model": model, "features": iris.feature_names, "targets": list(iris.target_names)},
    "model.pkl",
)
print("saved model.pkl")
```

```
model = RandomForestClassifier(...).fit(iris.data, iris.target)
```

붓꽃 데이터로 모델을 학습시킵니다. 이 한 번의 학습 결과를 저장할 거예요.

```
joblib.dump({"model": ..., "features": ..., "targets": ...}, "model.pkl")
```

모델뿐 아니라 **특성 이름·품종 이름**도 함께 묶어 저장합니다. 앱에서 슬라이더 라벨·결과 이름으로 바로 쓸 수 있어 편해요.

```
print("saved model.pkl")
```

저장이 끝나면 폴더에 **model.pkl** 파일이 생깁니다. 이제 앱이 이 파일을 불러 씁니다.

## STEP 2 예측 앱 (app.py) — 슬라이더 입력 → 예측

```
import streamlit as st
import pandas as pd
import joblib

@st.cache_resource
def load_bundle():
    return joblib.load("model.pkl")

b = load_bundle()
model, features, targets = b["model"], b["features"], b["targets"]

st.title("붓꽃 품종 예측기")
vals = [st.slider(f, 0.0, 8.0, 3.0, 0.1) for f in features]
X = pd.DataFrame([vals], columns=features)

if st.button("예측하기"):
    pred = model.predict(X)[0]
    st.success(f"예측 품종: {targets[pred]}")
```

@st.cache\_resource / load\_bundle()

model.pkl을 **한 번만** 불러와 재사용합니다(리런돼도 다시 안 읽음).

model, features, targets = b["model"], ...

저장할 때 함께 넣은 특성명·품종명을 꺼냅니다. 슬라이더와 결과에 그대로 활용해요.

vals = [st.slider(f, 0.0, 8.0, 3.0, 0.1) for f in features]

특성 4개마다 슬라이더를 자동으로 만들고, 그 값들을 리스트로 모읍니다.

X = pd.DataFrame([vals], columns=features)

입력값을 **학습 때와 같은 열 이름의 한 줄 표**로 만듭니다. 이게 예측의 필수 준비!

pred = model.predict(X)[0]

예측한 품종 번호를 받아, targets[pred] 로 이름을 보여줍니다.

### STEP 3 품 + 확률 시각화 — 완성형 예측기

```
import streamlit as st
import pandas as pd
import joblib

@st.cache_resource
def load_bundle():
    return joblib.load("model.pkl")

b = load_bundle()
model, features, targets = b["model"], b["features"], b["targets"]

st.title("붓꽃 품종 예측기")
with st.form("f"):
    vals = [st.slider(f, 0.0, 8.0, 3.0, 0.1) for f in features]
    submitted = st.form_submit_button("예측하기")

if submitted:
    X = pd.DataFrame([vals], columns=features)
    pred = model.predict(X)[0]
    proba = model.predict_proba(X)[0]
    st.success(f"예측 품종: {targets[pred]}")
    st.subheader("품종별 확률")
    st.bar_chart(pd.Series(proba, index=targets))
```

```
with st.form("f"): ... form_submit_button
```

슬라이더 4개를 폼으로 묶어 **예측하기**를 눌러야 한 번에 처리합니다(중간 리턴 방지).

```
proba = model.predict_proba(X)[0]
```

각 품종일 **확률**을 받습니다. 예측 결과에 확신 정도를 더해줘요.

```
st.bar_chart(pd.Series(proba, index=targets))
```

확률을 **막대 차트**로 시각화합니다. 품종 이름을 축 라벨로 붙여 한눈에 보이죠.

```
st.success(f"예측 품종: {targets[pred]}")
```

가장 확률이 높은 품종 이름을 초록 메시지로 강조합니다.



## Q9 전체 코드 한눈에

예측기의 폴더 구조와 두 파일( `train.py` · `app.py` ), 그리고 **실행 순서** 까지 한 번에. 복사해서 그대로 만들면 됩니다.

📁 폴더 구조 (streamlit\_app/)

```
streamlit_app/
├─ train.py      # 모델 학습 → model.pkl 생성
├─ app.py        # 예측 웹앱 (model.pkl 불러와 사용)
├─ model.pkl     # train.py 실행하면 자동 생성
├─ requirements.txt # 라이브러리 버전 고정
└─ docker-compose.yml # 도커 실행 설정
```

📄 train.py

```
import joblib
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier

iris = load_iris()
model = RandomForestClassifier(random_state=0).fit(iris.data, iris.target)

joblib.dump(
    {"model": model, "features": iris.feature_names, "targets": list(iris.target_names)},
    "model.pkl",
)
print("saved model.pkl")
```

```
import streamlit as st
import pandas as pd
import joblib

@st.cache_resource
def load_bundle():
    return joblib.load("model.pkl")

b = load_bundle()
model, features, targets = b["model"], b["features"], b["targets"]

st.title("붓꽃 품종 예측기")
with st.form("f"):
    vals = [st.slider(f, 0.0, 8.0, 3.0, 0.1) for f in features]
    submitted = st.form_submit_button("예측하기")

if submitted:
    X = pd.DataFrame([vals], columns=features)
    pred = model.predict(X)[0]
    proba = model.predict_proba(X)[0]
    st.success(f"예측 품종: {targets[pred]}")
    st.subheader("품종별 확률")
    st.bar_chart(pd.Series(proba, index=targets))
```

## ▶ 실행 순서 (터미널)

```
# 1) 모델 만들기 (맨 처음 한 번) → model.pkl 생성
python train.py
# 도커로 만들려면: docker compose run --rm streamlit python train.py

# 2) 앱 실행 → 브라우저에서 http://localhost:8601 접속
docker compose up
# 도커 없이 로컬이면: streamlit run app.py
```